

LABORATORY MANUAL

For

DATABASE MANAGEMENT SYSTEMS

Course Code: U25DSPC307

S.Y. BTECH

SEMESTER-III

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

PRACTICAL NO. 9

VIEWS AND QUERY PERFORMANCE ANALYSIS

1. Aim

To create and use views in MySQL and to analyse query execution plans using EXPLAIN and EXPLAIN ANALYZE, observing the effect of indexes on query performance.

2. Problem Statement

Using *college_db*:

1. Create a view that shows every student's result with department and course details, and list AIML students scoring 80 or more through it.
2. Create a department summary view (students and average marks) and check whether the views are updatable.
3. Create a view of Computer Engineering students that does not allow rows of other departments to be inserted through it.
4. Modify and drop views.
5. Create an attendance table with 100,000 records and find the number of absentees on 14 August 2025. Analyse the execution plan before and after creating a suitable index, and show how writing the condition affects the use of the index.

Note: The course document lists only the title of this practical. The case study, sample data and individual tasks below were prepared for this report to fulfil that title; they are illustrative and are not official university questions. All names, e-mail addresses and phone numbers are fictitious.

3. Objectives

1. To create simple, join and aggregate views and query them.
2. To understand updatable views and WITH CHECK OPTION.
3. To read execution plans produced by EXPLAIN, EXPLAIN FORMAT=TREE and EXPLAIN ANALYZE.
4. To measure the effect of an index on the rows examined and the execution time.
5. To write index-friendly (sargable) conditions.

4. Software / Tools Required

Sr. No.	Software / Tool	Purpose
1	MySQL Community Server 8.0 (version 8.0.46 used)	Relational DBMS on which all SQL statements were executed
2	MySQL command-line client (mysql)	Running the SQL scripts and recording the output
3	Operating system: Ubuntu 24.04 LTS (Windows 10/11 also suitable)	Platform for the DBMS server and client
4	Text editor (VS Code / Notepad++)	Writing and saving the .sql script files
5	MySQL Workbench 8.0 (optional)	Graphical (Visual Explain) view of plans

5. Theory

5.1 Views

A **view** is a named, stored SELECT query that behaves like a virtual table. A view does not store data (in MySQL); whenever it is used, its query is executed on the current base-table data. Views are used to:

- simplify complex queries (joins and aggregates are written once and reused);
- provide security - users can be granted access to a view that exposes only selected rows and columns, without access to the base tables (Practical 10);
- provide logical data independence - applications keep working when base tables are restructured, as long as the view is redefined.

```
CREATE [OR REPLACE] [ALGORITHM = {MERGE | TEMPTABLE | UNDEFINED}]
VIEW view_name [(column_list)] AS select_statement
[WITH [CASCADED | LOCAL] CHECK OPTION];
DROP VIEW [IF EXISTS] view_name;
```

Updatable views: INSERT, UPDATE and DELETE are allowed on a view only if each row of the view maps to exactly one row of a base table, i.e. the view has no aggregate functions, DISTINCT, GROUP BY, HAVING, UNION or subquery in the SELECT list. **WITH CHECK OPTION** rejects inserts and updates that would produce rows not satisfying the view's WHERE condition. MySQL processes a view either by *MERGE* (the view text is merged into the outer query) or *TEMPTABLE* (the view result is materialised in a temporary table).

5.2 Query Processing and Execution Plans

A SQL query is parsed, optimised and executed. The **cost-based optimizer** considers access paths (full table scan, index lookup, range scan) and join orders, estimates their cost from table statistics and chooses the cheapest **execution plan**. MySQL shows the plan with:

Statement	What it shows
EXPLAIN query	Estimated plan in tabular form, one row per table
EXPLAIN FORMAT=TREE query	Estimated plan as an iterator tree with costs (MySQL 8.0.16+)
EXPLAIN FORMAT=JSON query	Detailed estimated plan with cost information
EXPLAIN ANALYZE query	Executes the query and shows the tree with actual time, actual rows and loops for each step (MySQL 8.0.18+)
ANALYZE TABLE t	Refreshes the index statistics used by the optimizer

5.3 Important EXPLAIN Columns

Column	Meaning
type	Access type, from best to worst: system, const, eq_ref, ref, range, index (full index scan), ALL (full table scan)
possible_keys / key	Indexes that could be used / index actually chosen
key_len, ref	Bytes of the index used and the values compared with it
rows, filtered	Estimated rows examined and % of them satisfying the condition
Extra	Additional information, e.g. Using where, Using index (covering index)

5.4 Indexes

An index (a B+-tree in InnoDB) allows rows to be found without reading the whole table. A composite index (a, b) can be used for conditions on a, or on a and b (leftmost-prefix rule). An index whose columns contain all columns needed by a query is a *covering index*. A condition that applies a function to an indexed column (YEAR(col) = ...) is not *sargable* and cannot use an index range search. Indexes speed up reads but consume space and slow down INSERT, UPDATE and DELETE.

DBMS differences: PostgreSQL uses EXPLAIN and EXPLAIN (ANALYZE, BUFFERS); SQL Server uses SET SHOWPLAN_XML / 'Display Estimated Execution Plan' and SET STATISTICS TIME, IO; Oracle uses EXPLAIN PLAN FOR ... with DBMS_XPLAN.DISPLAY. MySQL Workbench can display MySQL plans graphically (Visual Explain).

6. Procedure / Algorithm

1. Create and populate *college_db* (Q1, Q2).
2. Create a view joining enrollment, student, department and course, and query it with WHERE and ORDER BY.
3. Create an aggregate view and use information_schema.views to check which views are updatable.
4. Create a view with WITH CHECK OPTION, insert and update through it, and observe the rejected operations.
5. Replace a view definition with CREATE OR REPLACE VIEW and drop a view.
6. Display the execution plan of a query on a view with EXPLAIN FORMAT=TREE.
7. Create a table with 100,000 rows using a recursive CTE and update its statistics.
8. Run a selective query and record its EXPLAIN and EXPLAIN ANALYZE output without a suitable index.
9. Create a composite index, repeat the analysis and compare access type, rows examined and actual time.
10. Compare a non-sargable condition with its sargable rewrite.

7. SQL Queries / Program

Q1: Setup: create college_db and its tables (from Practical 2)

```

DROP DATABASE IF EXISTS college_db;
CREATE DATABASE college_db;
USE college_db;

CREATE TABLE department (
    dept_id      INT            NOT NULL,
    dept_code    VARCHAR(10)    NOT NULL,
    dept_name    VARCHAR(40)    NOT NULL,
    building     VARCHAR(20),
    budget       DECIMAL(12,2)  DEFAULT 1000000.00,
    CONSTRAINT pk_department PRIMARY KEY (dept_id),
    CONSTRAINT uq_dept_code  UNIQUE (dept_code),
    CONSTRAINT chk_budget    CHECK (budget > 0)
);

CREATE TABLE instructor (
    instr_id      INT            NOT NULL,
    instr_name    VARCHAR(40)    NOT NULL,
    designation   VARCHAR(20)    NOT NULL,
    dept_id       INT,
    salary        DECIMAL(10,2)  NOT NULL,
    email         VARCHAR(40)    NOT NULL,
    mentor_id     INT,
    CONSTRAINT pk_instructor  PRIMARY KEY (instr_id),
    CONSTRAINT uq_instr_email UNIQUE (email),
    CONSTRAINT chk_salary    CHECK (salary BETWEEN 30000 AND 300000),
    CONSTRAINT fk_instr_dept FOREIGN KEY (dept_id)
        REFERENCES department (dept_id) ON DELETE SET NULL ON UPDATE CASCADE,
    CONSTRAINT fk_instr_mentor FOREIGN KEY (mentor_id)
        REFERENCES instructor (instr_id) ON DELETE SET NULL
);

CREATE TABLE student (
    student_id    INT            NOT NULL AUTO_INCREMENT,
    roll_no       VARCHAR(10)    NOT NULL,
    student_name  VARCHAR(40)    NOT NULL,
    gender        CHAR(1)        NOT NULL,
    dob           DATE           NOT NULL,
    dept_id       INT            NOT NULL,
    city          VARCHAR(30)    DEFAULT 'Pune',
    email         VARCHAR(40)    NOT NULL,
    CONSTRAINT pk_student    PRIMARY KEY (student_id),
    CONSTRAINT uq_roll_no    UNIQUE (roll_no),
    CONSTRAINT uq_student_email UNIQUE (email),
    CONSTRAINT chk_gender    CHECK (gender IN ('M', 'F', 'O')),
    CONSTRAINT chk_dob       CHECK (dob >= '1995-01-01'),
    CONSTRAINT fk_student_dept FOREIGN KEY (dept_id)
        REFERENCES department (dept_id) ON UPDATE CASCADE
);

CREATE TABLE student_phone (
    student_id    INT            NOT NULL,
    phone         CHAR(10)       NOT NULL,
    CONSTRAINT pk_student_phone PRIMARY KEY (student_id, phone),
    CONSTRAINT chk_phone      CHECK (phone REGEXP '^[6-9][0-9]{9}$'),
    CONSTRAINT fk_phone_student FOREIGN KEY (student_id)
        REFERENCES student (student_id) ON DELETE CASCADE
);

CREATE TABLE course (
    course_id     CHAR(5)        NOT NULL,
    title         VARCHAR(40)    NOT NULL,
    dept_id       INT            NOT NULL,

```

```

credits      TINYINT      NOT NULL,
instr_id     INT,
prereq_id    CHAR(5),
CONSTRAINT pk_course PRIMARY KEY (course_id),
CONSTRAINT uq_course_title UNIQUE (title),
CONSTRAINT chk_credits CHECK (credits BETWEEN 1 AND 5),
CONSTRAINT fk_course_dept FOREIGN KEY (dept_id)
    REFERENCES department (dept_id) ON UPDATE CASCADE,
CONSTRAINT fk_course_instr FOREIGN KEY (instr_id)
    REFERENCES instructor (instr_id) ON DELETE SET NULL,
CONSTRAINT fk_course_prereq FOREIGN KEY (prereq_id)
    REFERENCES course (course_id)
);

CREATE TABLE enrollment (
    student_id INT NOT NULL,
    course_id CHAR(5) NOT NULL,
    semester TINYINT NOT NULL,
    marks INT,
    grade CHAR(2),
    CONSTRAINT pk_enrollment PRIMARY KEY (student_id, course_id),
    CONSTRAINT chk_semester CHECK (semester BETWEEN 1 AND 8),
    CONSTRAINT chk_marks CHECK (marks BETWEEN 0 AND 100),
    CONSTRAINT fk_enr_student FOREIGN KEY (student_id)
        REFERENCES student (student_id) ON DELETE CASCADE,
    CONSTRAINT fk_enr_course FOREIGN KEY (course_id) REFERENCES course (course_id)
);

```

This setup script recreates the common *college_db* case-study database (schema of Practical 2 and data of Practical 3) so that this practical can be executed independently on a fresh MySQL server.

Q2: Setup: insert the sample data (from Practical 3)

```

INSERT INTO department (dept_id, dept_code, dept_name, building, budget) VALUES
(10, 'CSE', 'Computer Engineering', 'A Block', 2500000),
(20, 'AIML', 'AI and Machine Learning', 'B Block', 3000000),
(30, 'ECE', 'Electronics and Telecomm', 'C Block', 1800000),
(40, 'MECH', 'Mechanical Engineering', 'D Block', 1500000),
(50, 'CIVIL', 'Civil Engineering', 'D Block', 1200000);

INSERT INTO instructor VALUES
(101, 'Anil Kulkarni', 'Professor', 10, 185000, 'anil.k@example.com', NULL),
(102, 'Meera Joshi', 'Professor', 20, 172000, 'meera.j@example.com', 101),
(103, 'Rahul Patil', 'Asst. Prof', 10, 82000, 'rahul.p@example.com', 101),
(104, 'Sneha Deshmukh', 'Assoc. Prof', 20, 115000, 'sneha.d@example.com', 102),
(105, 'Vikram Shinde', 'Professor', 30, 168000, 'vikram.s@example.com', 101),
(106, 'Pooja Kale', 'Asst. Prof', 20, 78000, 'pooja.k@example.com', 104),
(107, 'Nitin Pawar', 'Asst. Prof', 50, 76000, 'nitin.p@example.com', 105),
(108, 'Kavita More', 'Assoc. Prof', 40, 112000, 'kavita.m@example.com', 105);

INSERT INTO student
(roll_no, student_name, gender, dob, dept_id, city, email) VALUES
('S24CS001', 'Aarav Sharma', 'M', '2005-03-14', 10, 'Pune', 'aarav@example.com'),
('S24CS002', 'Isha Gupta', 'F', '2005-07-22', 10, 'Mumbai', 'isha@example.com'),
('S24AI001', 'Rohan Mehta', 'M', '2005-01-09', 20, 'Pune', 'rohan@example.com'),
('S24AI002', 'Priya Nair', 'F', '2004-11-30', 20, 'Nashik', 'priya@example.com'),
('S24AI003', 'Kunal Verma', 'M', '2005-05-18', 20, 'Pune', 'kunal@example.com'),
('S24EC001', 'Sneha Iyer', 'F', '2005-09-02', 30, 'Nagpur', 'sneha@example.com'),
('S24EC002', 'Aditya Rao', 'M', '2004-12-25', 30, 'Mumbai', 'aditya@example.com'),
('S24ME001', 'Neha Kulkarni', 'F', '2005-02-11', 40, 'Pune', 'neha@example.com'),
('S24CS003', 'Yash Jadhav', 'M', '2005-08-05', 10, NULL, 'yash@example.com'),
('S24AI004', 'Ananya Desai', 'F', '2005-04-27', 20, 'Mumbai', 'ananya@example.com');

INSERT INTO student_phone VALUES
(1, '9876500011'), (1, '9876500012'), (2, '9876500021'), (3, '9876500031'),

```

```

(4, '9876500041'), (4, '9876500042'), (6, '9876500061'), (9, '9876500091');

INSERT INTO course VALUES
('CS101', 'Programming in C', 10, 4, 103, NULL),
('CS201', 'Data Structures', 10, 4, 103, 'CS101'),
('CS301', 'Database Management Systems', 10, 3, 101, 'CS201'),
('CS401', 'Advanced Databases', 10, 3, 101, 'CS301'),
('AI201', 'Python for Data Science', 20, 3, 104, 'CS101'),
('AI301', 'Machine Learning', 20, 4, 102, 'AI201'),
('AI401', 'Deep Learning', 20, 3, 106, 'AI301'),
('EC201', 'Digital Electronics', 30, 3, 105, NULL),
('EC301', 'Signals and Systems', 30, 4, NULL, 'EC201'),
('ME201', 'Engineering Mechanics', 40, 3, 108, NULL);

INSERT INTO enrollment (student_id, course_id, semester, marks, grade) VALUES
(1, 'CS101', 3, 78, 'A'), (1, 'CS201', 3, 85, 'A+'), (1, 'CS301', 4, 91, 'O'),
(1, 'AI201', 3, 72, 'A'), (2, 'CS101', 3, 88, 'A+'), (2, 'CS201', 3, 67, 'B+'),
(2, 'CS301', 4, 74, 'A'), (3, 'CS101', 3, 65, 'B+'), (3, 'AI201', 3, 81, 'A+'),
(3, 'AI301', 4, 89, 'A+'), (3, 'CS301', 4, 70, 'A'), (4, 'AI201', 3, 92, 'O'),
(4, 'AI301', 4, 76, 'A'), (4, 'AI401', 4, 84, 'A+'), (5, 'AI201', 3, 58, 'B'),
(5, 'AI301', 4, 63, 'B+'), (6, 'EC201', 3, 79, 'A'), (6, 'CS101', 3, 55, 'B'),
(6, 'EC301', 4, 82, 'A+'), (7, 'EC201', 3, 45, 'C'), (7, 'EC301', 4, 61, 'B+'),
(8, 'CS101', 3, 78, 'A'), (9, 'CS201', 3, 90, 'O'), (9, 'CS301', 4, 95, 'O'),
(9, 'CS401', 4, 88, 'A+');

```

Q3: Create a join view and query it like a table

```

CREATE VIEW v_student_result AS
SELECT s.roll_no, s.student_name, d.dept_code, c.course_id, c.title,
       e.semester, e.marks, e.grade
FROM   enrollment e
JOIN   student s   ON s.student_id = e.student_id
JOIN   department d ON d.dept_id = s.dept_id
JOIN   course c    ON c.course_id = e.course_id;

SELECT roll_no, student_name, course_id, marks, grade
FROM   v_student_result
WHERE  dept_code = 'AIML' AND marks >= 80
ORDER BY marks DESC;

```

Q4: Create an aggregate view and check which views are updatable

```

CREATE VIEW v_dept_summary AS
SELECT d.dept_code,
       COUNT(DISTINCT s.student_id) AS students,
       ROUND(AVG(e.marks), 2)      AS avg_marks
FROM   department d
LEFT JOIN student s   ON s.dept_id = d.dept_id
LEFT JOIN enrollment e ON e.student_id = s.student_id
GROUP BY d.dept_code;

SELECT * FROM v_dept_summary ORDER BY students DESC;

SELECT table_name, is_updatable, check_option
FROM   information_schema.views
WHERE  table_schema = 'college_db';

```

Q5: Updatable view WITH CHECK OPTION; DML through views

```

CREATE VIEW v_cse_student AS
SELECT student_id, roll_no, student_name, gender, dob, dept_id, email
FROM student
WHERE dept_id = 10
WITH CHECK OPTION;

INSERT INTO v_cse_student (roll_no, student_name, gender, dob, dept_id, email)
VALUES ('S24CS004', 'Om Patil', 'M', '2005-06-10', 10, 'om@example.com');
INSERT INTO v_cse_student (roll_no, student_name, gender, dob, dept_id, email)
VALUES ('S24AI005', 'Riya Shah', 'F', '2005-02-02', 20, 'riya@example.com');
UPDATE v_cse_student SET email = 'om.patil@example.com' WHERE roll_no = 'S24CS004';
SELECT roll_no, student_name, dept_id, email FROM v_cse_student;

UPDATE v_dept_summary SET students = 0 WHERE dept_code = 'CSE';

```

The second INSERT is rejected because the new row (dept_id 20) would not be visible through the view. The aggregate view v_dept_summary is not updatable because each of its rows is computed from many base rows.

Q6: Modify and drop views

```

CREATE OR REPLACE VIEW v_dept_summary AS
SELECT d.dept_code,
       COUNT(DISTINCT s.student_id) AS students,
       ROUND(AVG(e.marks), 2)      AS avg_marks,
       MAX(e.marks)                AS top_marks
FROM department d
LEFT JOIN student s ON s.dept_id = d.dept_id
LEFT JOIN enrollment e ON e.student_id = s.student_id
GROUP BY d.dept_code;
SELECT * FROM v_dept_summary WHERE students > 0;

DROP VIEW v_cse_student;
SHOW FULL TABLES WHERE table_type = 'VIEW';

```

Q7: Execution plan of a query on a view (EXPLAIN FORMAT=TREE)

```

EXPLAIN FORMAT=TREE
SELECT course_id, marks FROM v_student_result WHERE roll_no = 'S24CS001'\G

```

The plan contains only base tables, showing that MySQL *merged* the view definition into the query instead of materialising it. Because roll_no is UNIQUE, the student row (and its department row) were read as constant tables during optimisation, so the plan shows student_id = '1' as a constant and only the primary-key lookups on enrollment and course remain.

Q8: Create a large ATTENDANCE table (100,000 rows) for performance analysis

```

CREATE TABLE attendance (
    att_id      INT AUTO_INCREMENT PRIMARY KEY,
    student_id  INT      NOT NULL,
    course_id   CHAR(5)  NOT NULL,
    att_date    DATE      NOT NULL,
    status      CHAR(1)  NOT NULL      -- P = present, A = absent
);
SET SESSION cte_max_recursion_depth = 100000;
INSERT INTO attendance (student_id, course_id, att_date, status)
WITH RECURSIVE seq (n) AS (
    SELECT 1 UNION ALL SELECT n + 1 FROM seq WHERE n < 100000
)
SELECT 1 + n % 10,
       ELT(1 + n % 9, 'CS101', 'CS201', 'CS301', 'CS401', 'AI201',
                  'AI301', 'AI401', 'EC201', 'EC301'),
       DATE_ADD('2025-07-01', INTERVAL n % 180 DAY),
       IF(n % 7 = 0, 'A', 'P')
FROM   seq;
ANALYZE TABLE attendance;
SELECT COUNT(*) AS total_rows, MIN(att_date), MAX(att_date) FROM attendance;

```

Synthetic data: 10 students, 9 courses, 180 working days from 1 July 2025 and about one absence in seven records. Foreign keys are deliberately not declared on this log table, because InnoDB would automatically index the foreign key columns and hide the effect of the index created in Q10.

Q9: Query, EXPLAIN and EXPLAIN ANALYZE before creating an index

```

SELECT COUNT(*) AS absentees
FROM   attendance
WHERE  att_date = '2025-08-14' AND status = 'A';

EXPLAIN
SELECT COUNT(*) AS absentees
FROM   attendance
WHERE  att_date = '2025-08-14' AND status = 'A'\G

EXPLAIN ANALYZE
SELECT COUNT(*) AS absentees
FROM   attendance
WHERE  att_date = '2025-08-14' AND status = 'A'\G

```

Q10: Create a composite index and analyze the same query again

```

CREATE INDEX idx_att_date_status ON attendance (att_date, status);
SELECT index_name, seq_in_index, column_name, cardinality
FROM   information_schema.statistics
WHERE  table_schema = 'college_db' AND table_name = 'attendance';

SELECT COUNT(*) AS absentees
FROM   attendance
WHERE  att_date = '2025-08-14' AND status = 'A';

EXPLAIN
SELECT COUNT(*) AS absentees
FROM   attendance
WHERE  att_date = '2025-08-14' AND status = 'A'\G

EXPLAIN ANALYZE
SELECT COUNT(*) AS absentees
FROM   attendance
WHERE  att_date = '2025-08-14' AND status = 'A'\G

```

Q11: Effect of a function on an indexed column (sargable rewrite)

```
EXPLAIN SELECT COUNT(*) FROM attendance
WHERE YEAR(att_date) = 2025 AND MONTH(att_date) = 8\G

EXPLAIN SELECT COUNT(*) FROM attendance
WHERE att_date BETWEEN '2025-08-01' AND '2025-08-31'\G
```

Wrapping the column in YEAR()/MONTH() prevents an index range search, so MySQL scans the whole index (type: index, about 100,000 rows). Writing the same condition as a range on the bare column allows a range scan of only the August entries.

8. Sample Input / Database Tables

The following tables of *college_db* (created by Q1 and Q2) are used for the views. The values were read from the database after executing the setup queries.

Table: department

dept_id	dept_code	dept_name	building	budget
10	CSE	Computer Engineering	A Block	2500000.00
20	AIML	AI and Machine Learning	B Block	3000000.00
30	ECE	Electronics and Telecomm	C Block	1800000.00
40	MECH	Mechanical Engineering	D Block	1500000.00
50	CIVIL	Civil Engineering	D Block	1200000.00

Table: instructor

instr_id	instr_name	designation	dept_id	salary	email	mentor_id
101	Anil Kulkarni	Professor	10	185000.00	anil.k@example.com	NULL
102	Meera Joshi	Professor	20	172000.00	meera.j@example.com	101
103	Rahul Patil	Asst. Prof	10	82000.00	rahul.p@example.com	101
104	Sneha Deshmukh	Assoc. Prof	20	115000.00	sneha.d@example.com	102
105	Vikram Shinde	Professor	30	168000.00	vikram.s@example.com	101
106	Pooja Kale	Asst. Prof	20	78000.00	pooja.k@example.com	104
107	Nitin Pawar	Asst. Prof	50	76000.00	nitin.p@example.com	105
108	Kavita More	Assoc. Prof	40	112000.00	kavita.m@example.com	105

Table: student

student_id	roll_no	student_name	gender	dob	dept_id	city	email
1	S24CS001	Aarav Sharma	M	2005-03-14	10	Pune	aarav@example.com
2	S24CS002	Isha Gupta	F	2005-07-22	10	Mumbai	isha@example.com
3	S24AI001	Rohan Mehta	M	2005-01-09	20	Pune	rohan@example.com
4	S24AI002	Priya Nair	F	2004-11-30	20	Nashik	priya@example.com
5	S24AI003	Kunal Verma	M	2005-05-18	20	Pune	kunal@example.com
6	S24EC001	Sneha Iyer	F	2005-09-02	30	Nagpur	sneha@example.com
7	S24EC002	Aditya Rao	M	2004-12-25	30	Mumbai	aditya@example.com
8	S24ME001	Neha Kulkarni	F	2005-02-11	40	Pune	neha@example.com
9	S24CS003	Yash Jadhav	M	2005-08-05	10	NULL	yash@example.com
10	S24AI004	Ananya Desai	F	2005-04-27	20	Mumbai	ananya@example.com

Table: course

course_id	title	dept_id	credits	instr_id	prereq_id
AI201	Python for Data Science	20	3	104	CS101
AI301	Machine Learning	20	4	102	AI201
AI401	Deep Learning	20	3	106	AI301
CS101	Programming in C	10	4	103	NULL
CS201	Data Structures	10	4	103	CS101
CS301	Database Management Systems	10	3	101	CS201
CS401	Advanced Databases	10	3	101	CS301
EC201	Digital Electronics	30	3	105	NULL
EC301	Signals and Systems	30	4	NULL	EC201
ME201	Engineering Mechanics	40	3	108	NULL

Table: enrollment (25 rows)

student_id	course_id	semester	marks	grade
1	AI201	3	72	A
1	CS101	3	78	A
1	CS201	3	85	A+
1	CS301	4	91	O
2	CS101	3	88	A+
2	CS201	3	67	B+
2	CS301	4	74	A
3	AI201	3	81	A+
3	AI301	4	89	A+
3	CS101	3	65	B+

student_id	course_id	semester	marks	grade
3	CS301	4	70	A
4	AI201	3	92	O
4	AI301	4	76	A
4	AI401	4	84	A+
5	AI201	3	58	B
5	AI301	4	63	B+
6	CS101	3	55	B
6	EC201	3	79	A
6	EC301	4	82	A+
7	EC201	3	45	C
7	EC301	4	61	B+
8	CS101	3	78	A
9	CS201	3	90	O
9	CS301	4	95	O
9	CS401	4	88	A+

The ATTENDANCE table is generated by Q8; its first rows are shown below (read from the database after executing Q8).

Table: attendance (first 8 of 100,000 rows)

att_id	student_id	course_id	att_date	status
1	2	CS201	2025-07-02	P
2	3	CS301	2025-07-03	P
3	4	CS401	2025-07-04	P
4	5	AI201	2025-07-05	P
5	6	AI301	2025-07-06	P
6	7	AI401	2025-07-07	P
7	8	EC201	2025-07-08	A
8	9	EC301	2025-07-09	P

9. Output

The output below is the actual text produced by the MySQL 8.0.46 command-line client when the statements of Section 7 were executed in batch mode (`mysql --table -vvv < script.sql`). Each result is preceded by the statement that produced it; statements longer than a few lines are abbreviated with '...'. Because the script ran in batch mode, error messages report the line number within the script file. Execution times shown in brackets depend on the machine. The actual times reported by `EXPLAIN ANALYZE` also vary from run to run.

Output of Q1: Setup: create college_db and its tables (from Practical 2)

```
mysql> DROP DATABASE IF EXISTS college_db;
Query OK, 0 rows affected, 1 warning (0.00 sec)

mysql> CREATE DATABASE college_db;
Query OK, 1 row affected (0.00 sec)

mysql> CREATE TABLE department ( ...;
Query OK, 0 rows affected (0.02 sec)

mysql> CREATE TABLE instructor ( ...;
Query OK, 0 rows affected (0.02 sec)

mysql> CREATE TABLE student ( ...;
Query OK, 0 rows affected (0.02 sec)

mysql> CREATE TABLE student_phone ( ...;
Query OK, 0 rows affected (0.01 sec)

mysql> CREATE TABLE course ( ...;
Query OK, 0 rows affected (0.03 sec)

mysql> CREATE TABLE enrollment ( ...;
Query OK, 0 rows affected (0.02 sec)
```

Output of Q2: Setup: insert the sample data (from Practical 3)

```
mysql> INSERT INTO department (dept_id, dept_code, dept_name, building, budget) VALUES ...;
Query OK, 5 rows affected (0.00 sec)
Records: 5 Duplicates: 0 Warnings: 0

mysql> INSERT INTO instructor VALUES ...;
Query OK, 8 rows affected (0.00 sec)
Records: 8 Duplicates: 0 Warnings: 0

mysql> INSERT INTO student ...;
Query OK, 10 rows affected (0.00 sec)
Records: 10 Duplicates: 0 Warnings: 0

mysql> INSERT INTO student_phone VALUES ...;
Query OK, 8 rows affected (0.01 sec)
Records: 8 Duplicates: 0 Warnings: 0

mysql> INSERT INTO course VALUES ...;
Query OK, 10 rows affected (0.00 sec)
Records: 10 Duplicates: 0 Warnings: 0

mysql> INSERT INTO enrollment (student_id, course_id, semester, marks, grade) VALUES ...;
Query OK, 25 rows affected (0.01 sec)
Records: 25 Duplicates: 0 Warnings: 0
```

Output of Q3: Create a join view and query it like a table

```
mysql> CREATE VIEW v_student_result AS
-> SELECT s.roll_no, s.student_name, d.dept_code, c.course_id, c.title,
->         e.semester, e.marks, e.grade
-> FROM   enrollment e
-> JOIN   student s    ON s.student_id = e.student_id
-> JOIN   department d ON d.dept_id = s.dept_id
-> JOIN   course c     ON c.course_id = e.course_id;
Query OK, 0 rows affected (0.00 sec)

mysql> SELECT roll_no, student_name, course_id, marks, grade
-> FROM   v_student_result
-> WHERE  dept_code = 'AIML' AND marks >= 80
-> ORDER BY marks DESC;
+-----+-----+-----+-----+-----+
| roll_no | student_name | course_id | marks | grade |
+-----+-----+-----+-----+-----+
| S24AI002 | Priya Nair   | AI201     | 92    | 0      |
| S24AI001 | Rohan Mehta  | AI301     | 89    | A+     |
| S24AI002 | Priya Nair   | AI401     | 84    | A+     |
| S24AI001 | Rohan Mehta  | AI201     | 81    | A+     |
+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)
```

Output of Q4: Create an aggregate view and check which views are updatable

```
mysql> CREATE VIEW v_dept_summary AS
-> SELECT d.dept_code,
->        COUNT(DISTINCT s.student_id) AS students,
->        ROUND(AVG(e.marks), 2)      AS avg_marks
-> FROM   department d
-> LEFT JOIN student s    ON s.dept_id = d.dept_id
-> LEFT JOIN enrollment e ON e.student_id = s.student_id
-> GROUP BY d.dept_code;
Query OK, 0 rows affected (0.00 sec)

mysql> SELECT * FROM v_dept_summary ORDER BY students DESC;
+-----+-----+-----+
| dept_code | students | avg_marks |
+-----+-----+-----+
| AIML      | 4        | 75.33     |
| CSE       | 3        | 82.80     |
| ECE       | 2        | 64.40     |
| MECH      | 1        | 78.00     |
| CIVIL     | 0        | NULL      |
+-----+-----+-----+
5 rows in set (0.00 sec)

mysql> SELECT table_name, is_updatable, check_option
-> FROM   information_schema.views
-> WHERE  table_schema = 'college_db';
+-----+-----+-----+
| TABLE_NAME | IS_UPDATABLE | CHECK_OPTION |
+-----+-----+-----+
| v_dept_summary | NO          | NONE         |
| v_student_result | YES        | NONE         |
+-----+-----+-----+
2 rows in set (0.00 sec)
```

Output of Q5: Updatable view WITH CHECK OPTION; DML through views

```
mysql> CREATE VIEW v_cse_student AS
-> SELECT student_id, roll_no, student_name, gender, dob, dept_id, email
-> FROM   student
-> WHERE  dept_id = 10
-> WITH CHECK OPTION;
Query OK, 0 rows affected (0.01 sec)

mysql> INSERT INTO v_cse_student (roll_no, student_name, gender, dob, dept_id, email)
```

```

-> VALUES ('S24CS004', 'Om Patil', 'M', '2005-06-10', 10, 'om@example.com');
Query OK, 1 row affected (0.00 sec)

mysql> INSERT INTO v_cse_student (roll_no, student_name, gender, dob, dept_id, email)
-> VALUES ('S24AI005', 'Riya Shah', 'F', '2005-02-02', 20, 'riya@example.com');
ERROR 1369 (HY000) at line 185: CHECK OPTION failed 'college_db.v_cse_student'

mysql> UPDATE v_cse_student SET email = 'om.patil@example.com' WHERE roll_no = 'S24CS004';
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> SELECT roll_no, student_name, dept_id, email FROM v_cse_student;
+-----+-----+-----+-----+
| roll_no | student_name | dept_id | email |
+-----+-----+-----+-----+
| S24CS001 | Aarav Sharma | 10 | aarav@example.com |
| S24CS002 | Isha Gupta | 10 | isha@example.com |
| S24CS003 | Yash Jadhav | 10 | yash@example.com |
| S24CS004 | Om Patil | 10 | om.patil@example.com |
+-----+-----+-----+-----+
4 rows in set (0.01 sec)

mysql> UPDATE v_dept_summary SET students = 0 WHERE dept_code = 'CSE';
ERROR 1288 (HY000) at line 190: The target table v_dept_summary of the UPDATE is not updatable

```

Output of Q6: Modify and drop views

```

mysql> CREATE OR REPLACE VIEW v_dept_summary AS
-> SELECT d.dept_code,
->         COUNT(DISTINCT s.student_id) AS students,
->         ROUND(AVG(e.marks), 2)      AS avg_marks,
->         MAX(e.marks)                 AS top_marks
-> FROM    department d
-> LEFT JOIN student s    ON s.dept_id = d.dept_id
-> LEFT JOIN enrollment e ON e.student_id = s.student_id
-> GROUP BY d.dept_code;
Query OK, 0 rows affected (0.00 sec)

mysql> SELECT * FROM v_dept_summary WHERE students > 0;
+-----+-----+-----+-----+
| dept_code | students | avg_marks | top_marks |
+-----+-----+-----+-----+
| AIML     | 4 | 75.33 | 92 |
| CSE      | 4 | 82.80 | 95 |
| ECE      | 2 | 64.40 | 82 |
| MECH     | 1 | 78.00 | 78 |
+-----+-----+-----+-----+
4 rows in set (0.00 sec)

mysql> DROP VIEW v_cse_student;
Query OK, 0 rows affected (0.00 sec)

mysql> SHOW FULL TABLES WHERE table_type = 'VIEW';
+-----+-----+
| Tables_in_college_db | Table_type |
+-----+-----+
| v_dept_summary       | VIEW      |
| v_student_result     | VIEW      |
+-----+-----+
2 rows in set (0.00 sec)

```

Output of Q7: Execution plan of a query on a view (EXPLAIN FORMAT=TREE)

```
mysql> EXPLAIN FORMAT=TREE
-> SELECT course_id, marks FROM v_student_result WHERE roll_no = 'S24CS001'\G
***** 1. row *****
EXPLAIN: -> Nested loop inner join (cost=2.05 rows=4)
-> Index lookup on e using PRIMARY (student_id='1') (cost=0.651 rows=4)
-> Single-row covering index lookup on c using PRIMARY (course_id=e.course_id)
(cost=0.275 rows=1)

1 row in set (0.00 sec)
```

Output of Q8: Create a large ATTENDANCE table (100,000 rows) for performance analysis

```
mysql> CREATE TABLE attendance (
->   att_id      INT AUTO_INCREMENT PRIMARY KEY,
->   student_id  INT      NOT NULL,
->   course_id   CHAR(5) NOT NULL,
->   att_date    DATE     NOT NULL,
->   status      CHAR(1) NOT NULL
-> );
Query OK, 0 rows affected (0.01 sec)

mysql> SET SESSION cte_max_recursion_depth = 100000;
Query OK, 0 rows affected (0.00 sec)

mysql> INSERT INTO attendance (student_id, course_id, att_date, status)
-> WITH RECURSIVE seq (n) AS (
->   SELECT 1 UNION ALL SELECT n + 1 FROM seq WHERE n < 100000
-> )
-> ...
-> FROM seq;
Query OK, 100000 rows affected (0.68 sec)
Records: 100000 Duplicates: 0 Warnings: 0

mysql> ANALYZE TABLE attendance;
+-----+-----+-----+-----+
| Table                | Op      | Msg_type | Msg_text |
+-----+-----+-----+-----+
| college_db.attendance | analyze | status   | OK        |
+-----+-----+-----+-----+
1 row in set (0.00 sec)

mysql> SELECT COUNT(*) AS total_rows, MIN(att_date), MAX(att_date) FROM attendance;
+-----+-----+-----+
| total_rows | MIN(att_date) | MAX(att_date) |
+-----+-----+-----+
|      100000 | 2025-07-01    | 2025-12-27    |
+-----+-----+-----+
1 row in set (0.02 sec)
```

Output of Q9: Query, EXPLAIN and EXPLAIN ANALYZE before creating an index

```
mysql> SELECT COUNT(*) AS absentees
-> FROM attendance
-> WHERE att_date = '2025-08-14' AND status = 'A';
+-----+
| absentees |
+-----+
|          80 |
+-----+
1 row in set (0.02 sec)

mysql> EXPLAIN
-> SELECT COUNT(*) AS absentees
-> FROM attendance
-> WHERE att_date = '2025-08-14' AND status = 'A'\G
***** 1. row *****
id: 1
```

```

select_type: SIMPLE
table: attendance
partitions: NULL
type: ALL
possible_keys: NULL
key: NULL
key_len: NULL
ref: NULL
rows: 97527
filtered: 1.00
Extra: Using where
1 row in set, 1 warning (0.00 sec)

mysql> EXPLAIN ANALYZE
-> SELECT COUNT(*) AS absentees
-> FROM attendance
-> WHERE att_date = '2025-08-14' AND status = 'A'\G
***** 1. row *****
EXPLAIN: -> Aggregate: count(0) (cost=9922 rows=1) (actual time=23.2..23.2 rows=1 loops=1)
-> Filter: ((attendance.`status` = 'A') and (attendance.att_date = DATE'2025-08-14'))
(cost=9825 rows=975) (actual time=0.684..23.2 rows=80 loops=1)
-> Table scan on attendance (cost=9825 rows=97527) (actual time=0.652..16.2
rows=100000 loops=1)

1 row in set (0.03 sec)

```

Output of Q10: Create a composite index and analyze the same query again

```

mysql> CREATE INDEX idx_att_date_status ON attendance (att_date, status);
Query OK, 0 rows affected (0.23 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> SELECT index_name, seq_in_index, column_name, cardinality
-> FROM information_schema.statistics
-> WHERE table_schema = 'college_db' AND table_name = 'attendance';
+-----+-----+-----+-----+
| INDEX_NAME          | SEQ_IN_INDEX | COLUMN_NAME | CARDINALITY |
+-----+-----+-----+-----+
| idx_att_date_status | 1            | att_date   | 180         |
| idx_att_date_status | 2            | status     | 364         |
| PRIMARY             | 1            | att_id     | 97527       |
+-----+-----+-----+-----+
3 rows in set (0.00 sec)

mysql> SELECT COUNT(*) AS absentees
-> FROM attendance
-> WHERE att_date = '2025-08-14' AND status = 'A';
+-----+
| absentees |
+-----+
| 80        |
+-----+
1 row in set (0.00 sec)

mysql> EXPLAIN
-> SELECT COUNT(*) AS absentees
-> FROM attendance
-> WHERE att_date = '2025-08-14' AND status = 'A'\G
***** 1. row *****
id: 1
select_type: SIMPLE
table: attendance
partitions: NULL
type: ref
possible_keys: idx_att_date_status
key: idx_att_date_status
key_len: 7
ref: const,const
rows: 80
filtered: 100.00
Extra: Using where; Using index
1 row in set, 1 warning (0.00 sec)

```

```
mysql> EXPLAIN ANALYZE
-> SELECT COUNT(*) AS absentees
-> FROM attendance
-> WHERE att_date = '2025-08-14' AND status = 'A'\G
***** 1. row *****
EXPLAIN: -> Aggregate: count(0) (cost=17.1 rows=1) (actual time=0.0566..0.0567 rows=1
loops=1)
-> Filter: (attendance.`status` = 'A') (cost=9.11 rows=80) (actual time=0.0322..0.0506
rows=80 loops=1)
-> Covering index lookup on attendance using idx_att_date_status
(att_date=DATE'2025-08-14', status='A') (cost=9.11 rows=80) (actual time=0.0311..0.0405
rows=80 loops=1)

1 row in set (0.00 sec)
```

Output of Q11: Effect of a function on an indexed column (sargable rewrite)

```
mysql> EXPLAIN SELECT COUNT(*) FROM attendance
-> WHERE YEAR(att_date) = 2025 AND MONTH(att_date) = 8\G
***** 1. row *****
      id: 1
  select_type: SIMPLE
        table: attendance
  partitions: NULL
        type: index
possible_keys: NULL
         key: idx_att_date_status
        key_len: 7
         ref: NULL
        rows: 97527
   filtered: 100.00
    Extra: Using where; Using index
1 row in set, 1 warning (0.00 sec)

mysql> EXPLAIN SELECT COUNT(*) FROM attendance
-> WHERE att_date BETWEEN '2025-08-01' AND '2025-08-31'\G
***** 1. row *****
      id: 1
  select_type: SIMPLE
        table: attendance
  partitions: NULL
        type: range
possible_keys: idx_att_date_status
         key: idx_att_date_status
        key_len: 3
         ref: NULL
        rows: 32578
   filtered: 100.00
    Extra: Using where; Using index
1 row in set, 1 warning (0.00 sec)
```

10. Result

Join, aggregate and CHECK OPTION views were created and used; information_schema confirmed that the join and simple views are updatable while the aggregate view is not, and MySQL rejected an out-of-range insert (ERROR 1369) and an update of the aggregate view (ERROR 1288). EXPLAIN showed that the view was merged into the query. For the attendance query, the plan without an index was a full table scan (type ALL, about 100,000 rows examined); after creating the composite index idx_att_date_status the plan changed to an index lookup (type ref) examining only the matching rows, and EXPLAIN ANALYZE reported a much lower actual time. A function on the indexed column forced a full index scan, whereas the equivalent range condition used a range scan.

11. Conclusion

Views provide a simple, reusable and secure interface to complex queries without storing extra data, and WITH CHECK OPTION keeps DML through a view consistent with its definition. EXPLAIN and EXPLAIN ANALYZE make the optimizer's decisions visible, so that slow queries can be diagnosed from the access type and the number of rows examined. A well-chosen composite index reduced the work from scanning the whole table to reading only the matching index entries, but only when the query condition was written in a sargable form.